

Cross Relation Informed Cryptocurrency Price Prediction

Meriç Demirörs (2841283)
[https://github.com/mericdemirors/
CrossCurrencyPrediction](https://github.com/mericdemirors/CrossCurrencyPrediction)

Try it out with interactive plots→



MAIN FLOW OF THE PROJECT

- 1- Collect data
- 2- Prepare the data for models' use
- 3- Train models to find "trainable" ones
- 4- Find the best ones that are the most "profitable"
- 5- Perfect the most "profitable" model

DATA

Time

2018-05-04
06:00:00

2024-08-21
18:00:00

2025-05-04
18:00:00

Training Portion (2304 days)

Validation Portion
(256 days)

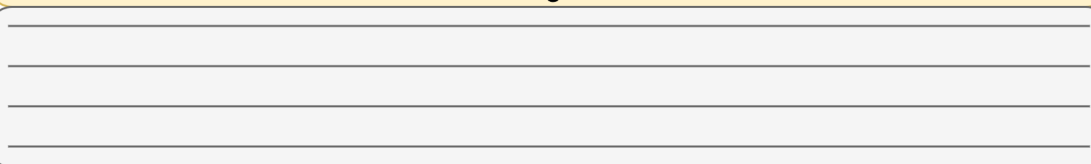
Testing
Portion

Time

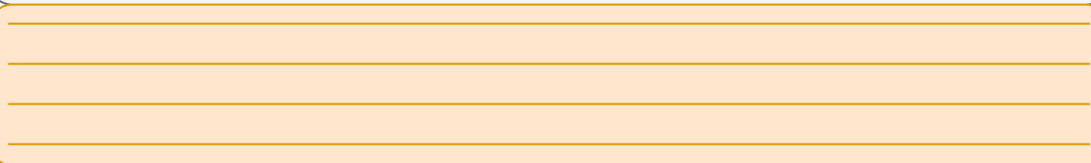
BTC

9635.39	9670.39	...	open
9670.00	9665.00	...	close
9605.00	9585.00	...	low
9830.04	9860.42	...	high

ETH



BNB



XRP



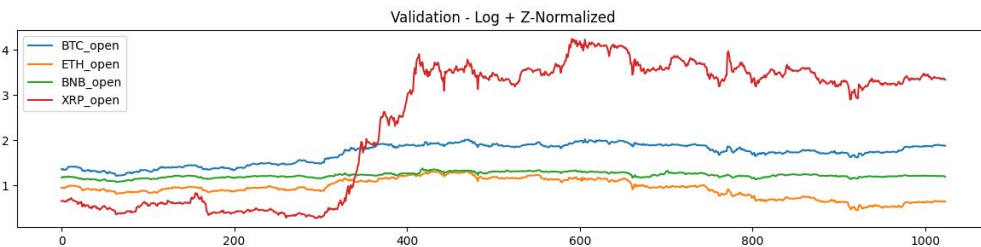
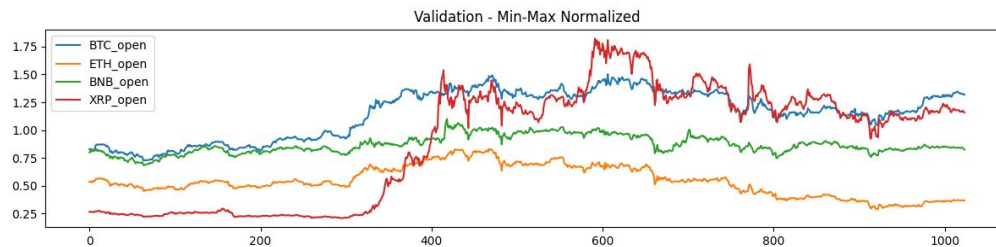
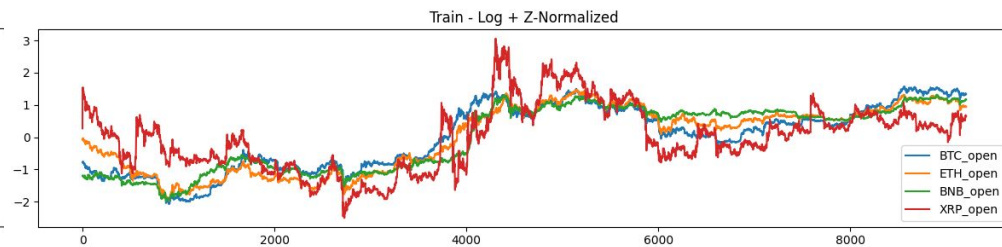
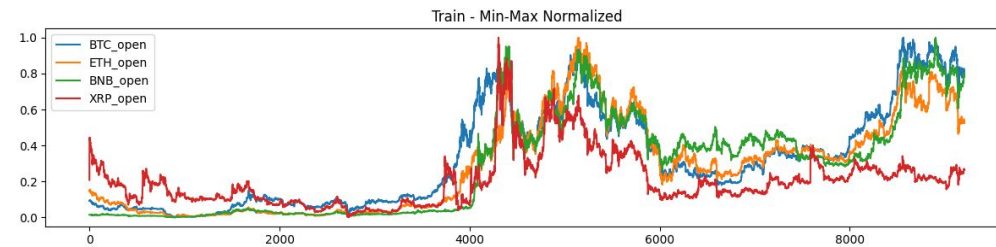
DATA

Time →			
BTC	9635.39	9670.39	...
	9670.00	9665.00	...
	9605.00	9585.00	...
	9830.04	9860.42	...
			open
			close
			low
			high
ETH			
			INPUT
BNB			
XRP			

Why not directly
do this?

- 1- Huge prices
are not suitable
to work with
- 2- Values should
be similar across
coins

DATA

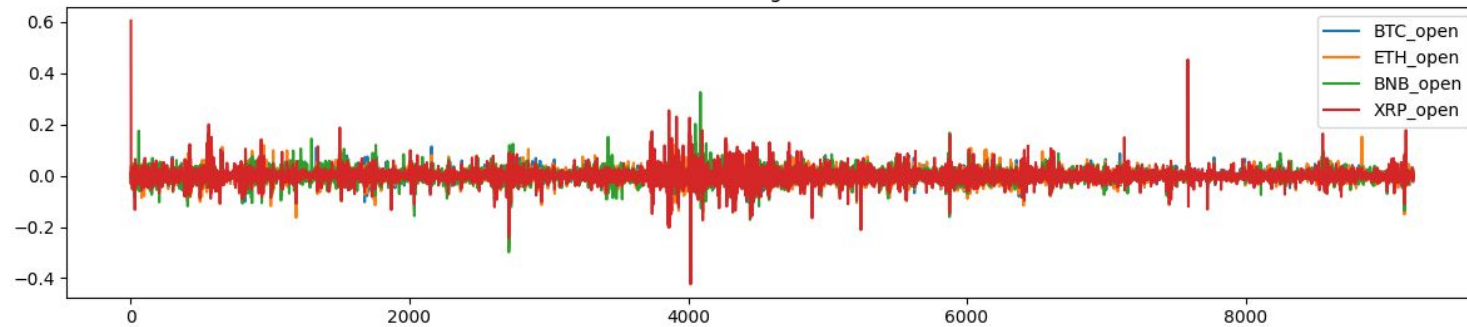


Basic normalizations won't work because of the domain shift between the train and validation. We need some time-irrelevant normalization, like "Log Returns".

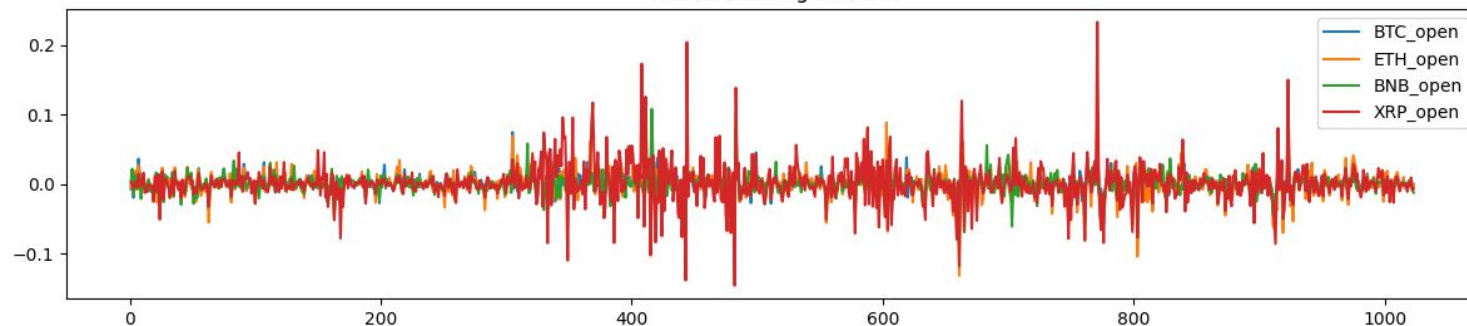
DATA

Log Returns: $\log(\text{today}/\text{yesterday})$

Train - Log Returns



Validation - Log Returns



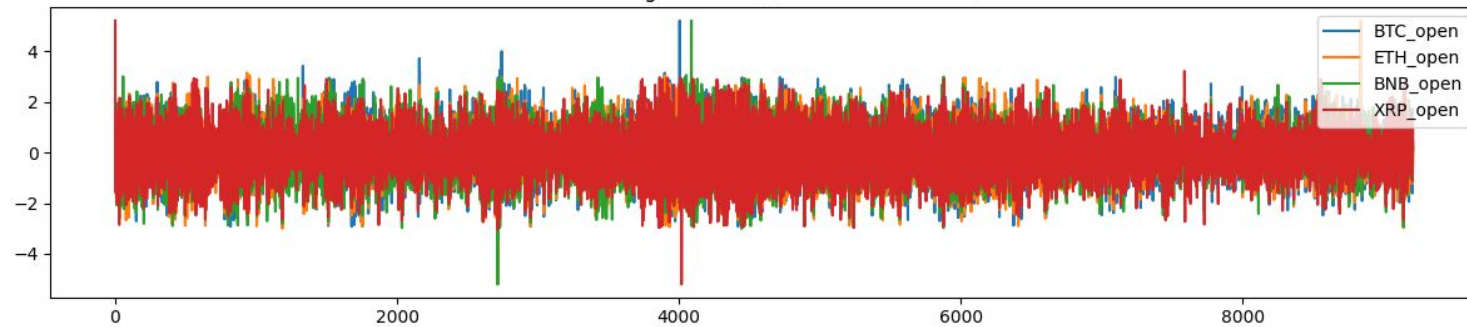
Still, not perfect.

All coins are almost in the same range except few peaks. But domain shift is not solved.

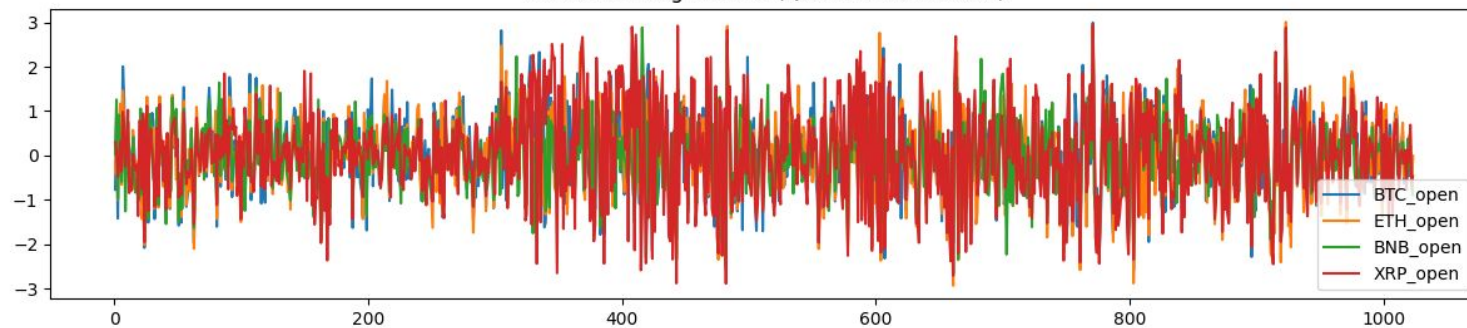
DATA

+ `sklearn.preprocessing.QuantileTransformer`

Train - Log Returns (Quantile Normalized)

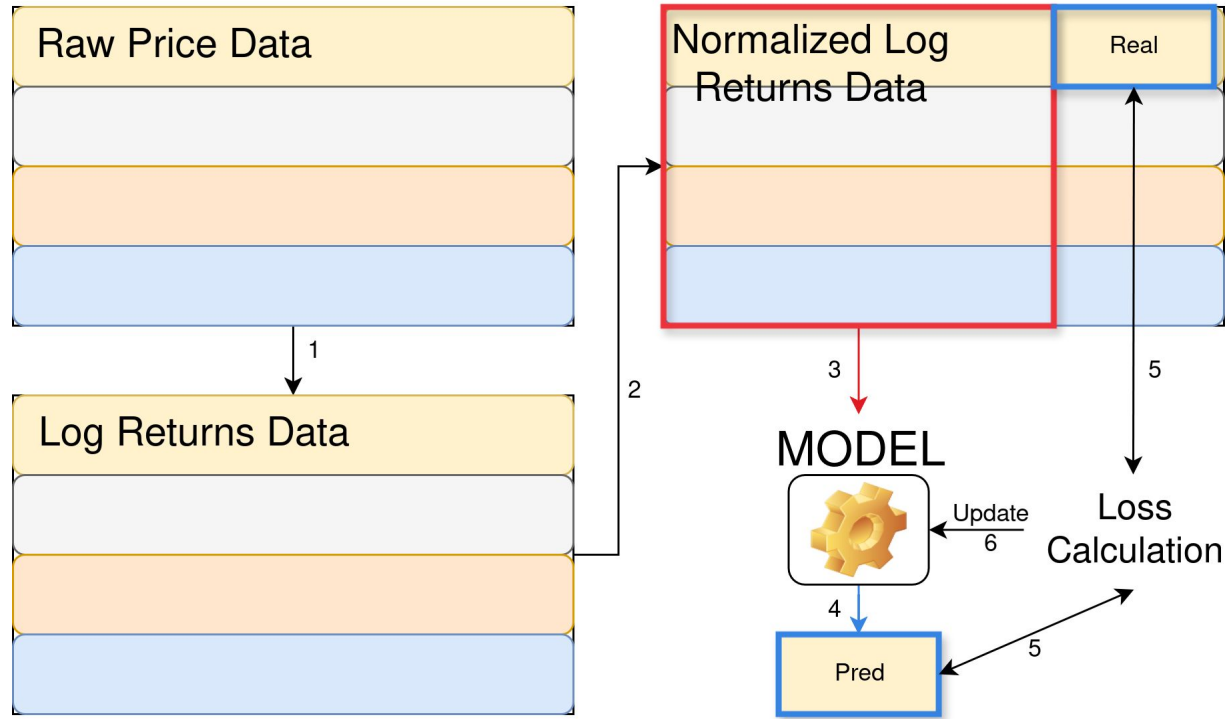


Validation - Log Returns (Quantile Normalized)



Now data is
more 'normal'
for the models.

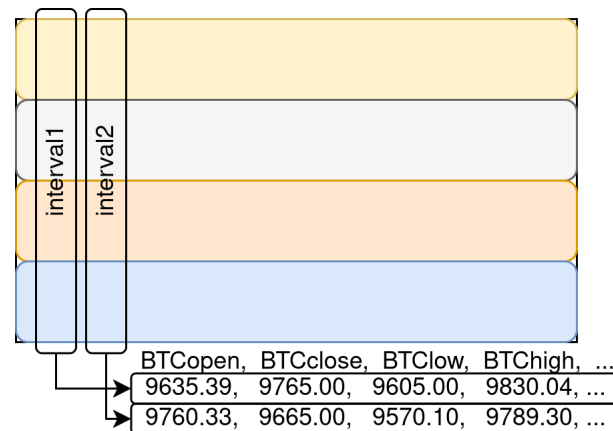
DATA



We do everything in the normalized log returns space. Then de-normalize the predictions with reverse transform and reversing log returns to get predicted raw prices.

DATASETS

I tried different datasets, each emphasizing a different aspect of the data:

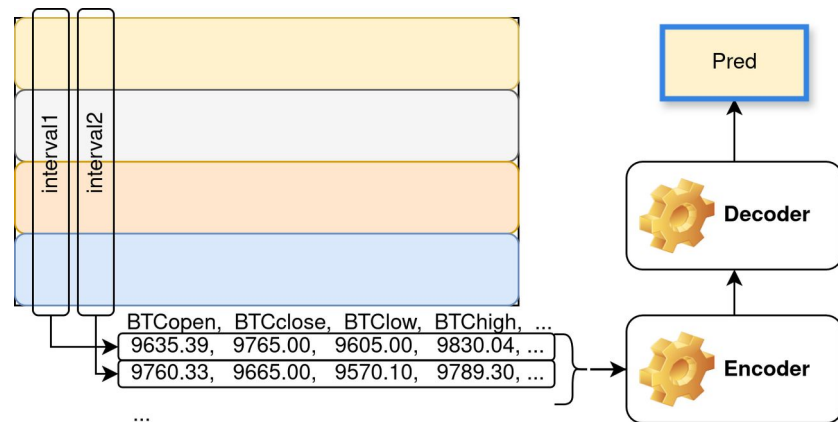


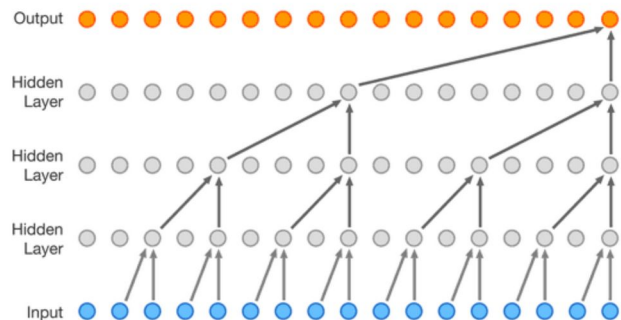
- 1. LogReturnTransformCoinDataset:** regular log returns, calculates $\log(\text{interval2}/\text{interval1})$. Then normalized.
- 2. IntervalLogReturnTransformCoinDataset:** calculates $\log(\text{interval2}/\text{interval1})$ only for the "open" feature, and $\log(\text{feature}/\text{open})$ per-coin inside the interval. Then normalized.
- 3. IntervalLogReturnTransformLowHighRootCoinDataset:** same with the previous one, but instead of normalizing low and high features, it does $\text{root}(\text{feature}) * \text{sign}(\text{feature})$ to encode the logic of "low $\leq$ open & high $\geq$ open" into the normalization. Then remaining features are normalized.
- 4. LogReturnTransformPriceLossCoinDataset:** same with the first one, adds additional loss on the raw price difference. (Using big prices on loss calculation resulted in instability, not used further)
- 5. MergedIntervalsLogReturnTransformCoinDataset:** same with the first one, but concatenates intervals to enrich the interval content, + allows to utilize low-high order in the concatenated intervals. (Enriching interval content didn't make a big change, + implementing low-high order didn't worked more on it later as additional information.)

MODELS

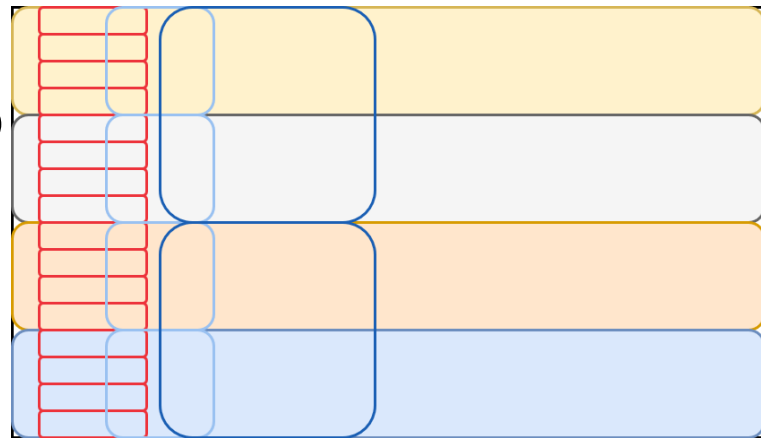
I have 3 different models for this time-series data. All follow basic sequence-to-sequence encoder-decoder architecture (with some little details I'm skipping for the time limit):

1. EncoderDecoderGRU
2. EncoderDecoderLSTM
3. EncoderDecoderTransformer





MODELS

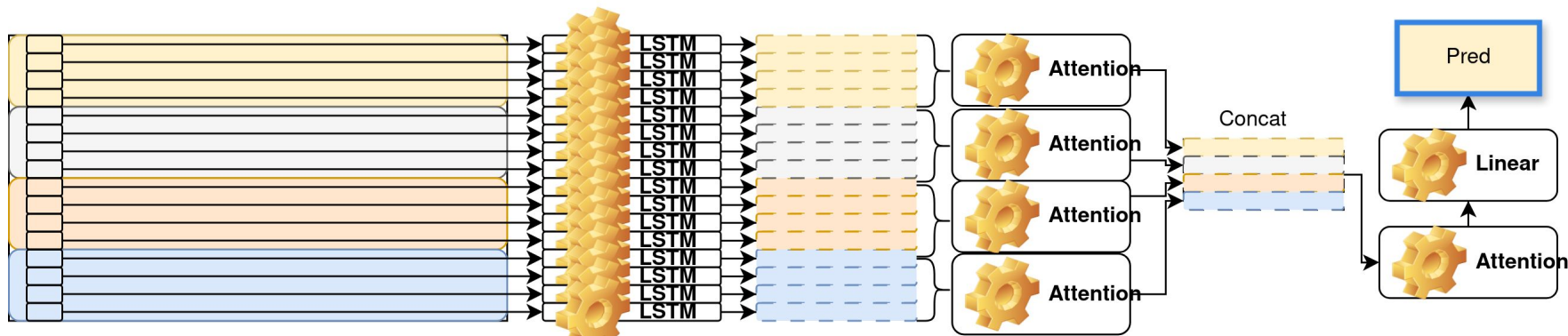
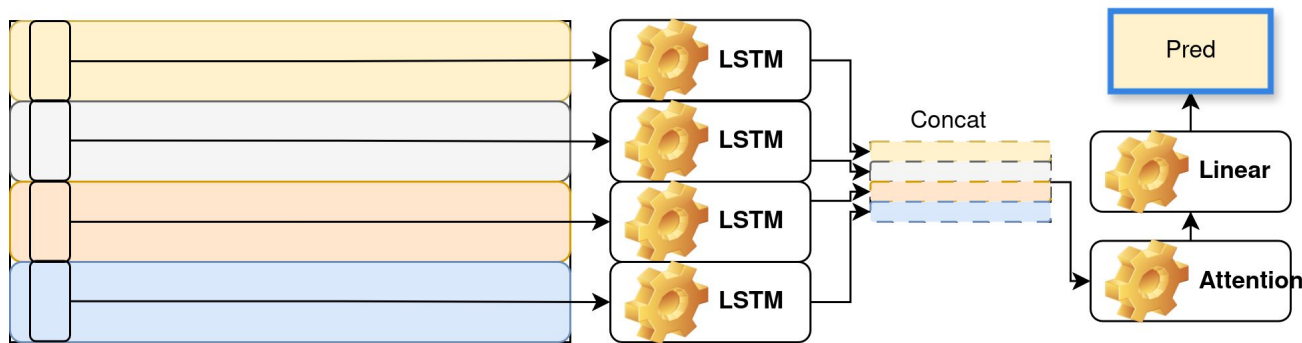


I also wanted to implement a convolutional model similar to how Wavenet processes time series data, so I implemented TCN, which starts with 1D kernels (red) for feature-wise processing and then switches to 2D kernels (blues) for bigger scoped processing.

MODELS

This idea of first processing features and then looking into bigger parts inspired 2 more model architectures:

CoinWiseCrossAttentionLSTM
FeatureWiseCrossAttentionLSTM

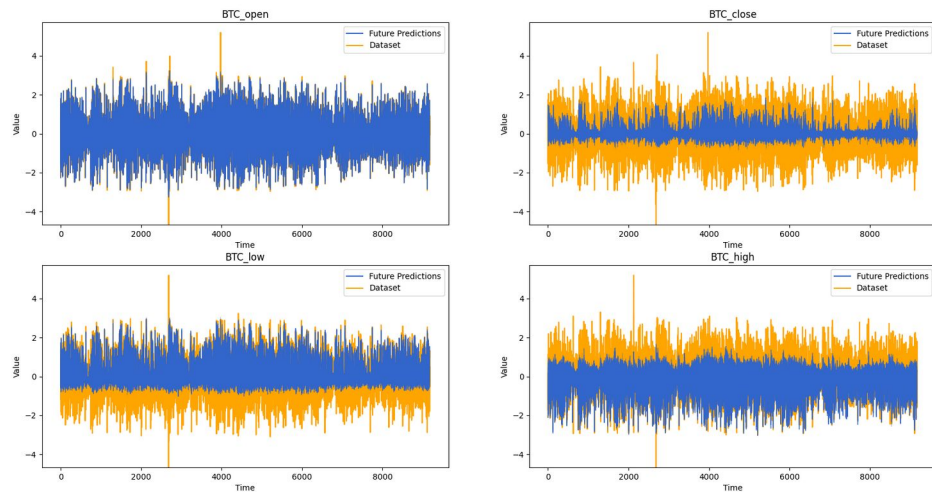


TRAINING: PHASE 1

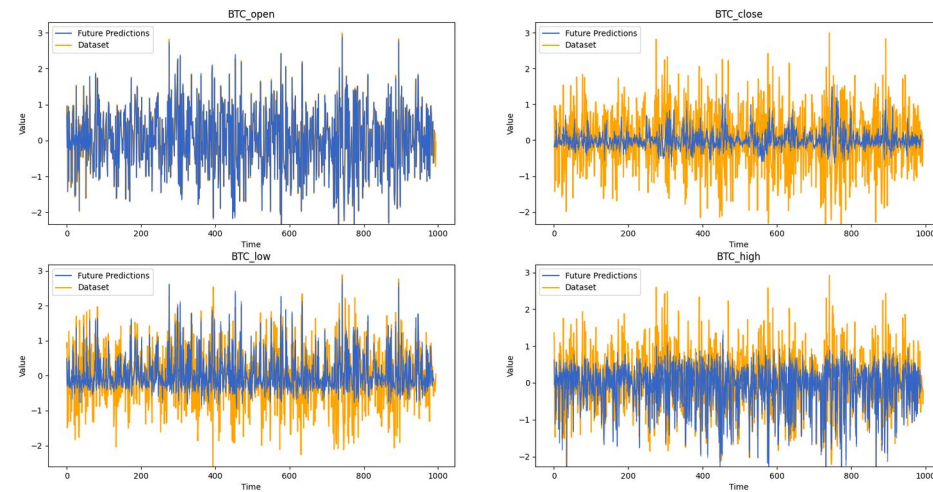
Now we have 6 models: GRU, LSTM, Transformer, TCN, Coin and Feature attention models. Each of them trained and evaluated with the datasets to find good pairs.

Important note: there are multiple important aspects to consider if a pair is good or bad. Our goal is to make money at the end but we can't evaluate solely on that, so in the first phase we are looking for "trainable" models. To find trainable models, we need to look more than just the loss. Here are some stuff we need to check for a good pair:

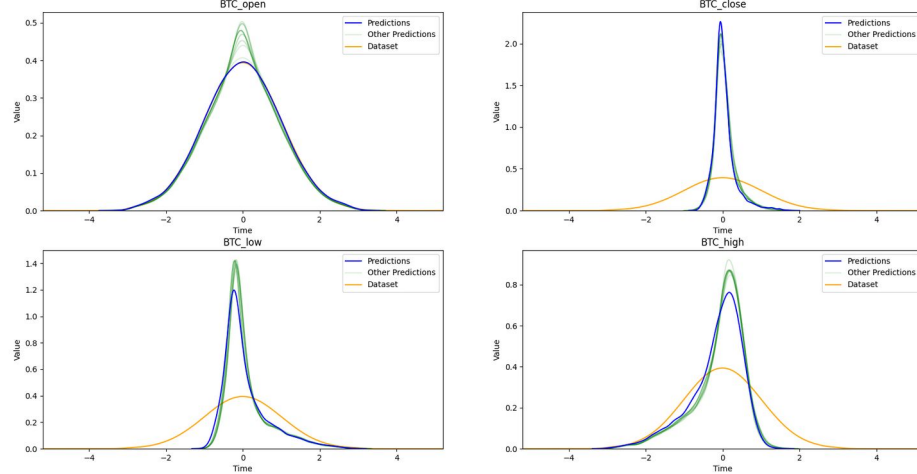
Predictions on the train dataset.
Blue: price_t+1 from price_t where $i > 0$, fading away for further predictions



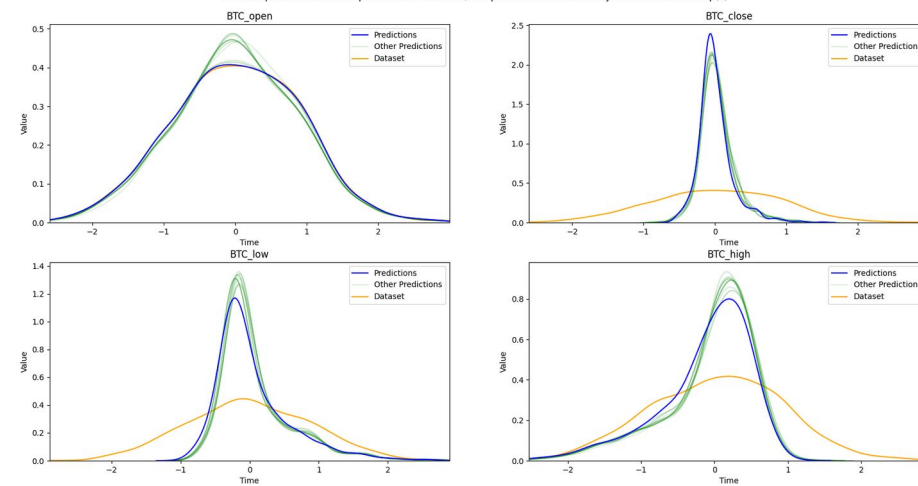
Predictions on the val dataset.
Blue: price_t+1 from price_t where $i > 0$, fading away for further predictions



Distributions of predictions on the train dataset.
Blue: price_t+1 from price_t, Greens: price_t+1 from price_t where $i > 1$
All other plots check the temporal fit to the dataset, this plot checks overall ability to match real world $p(x)$

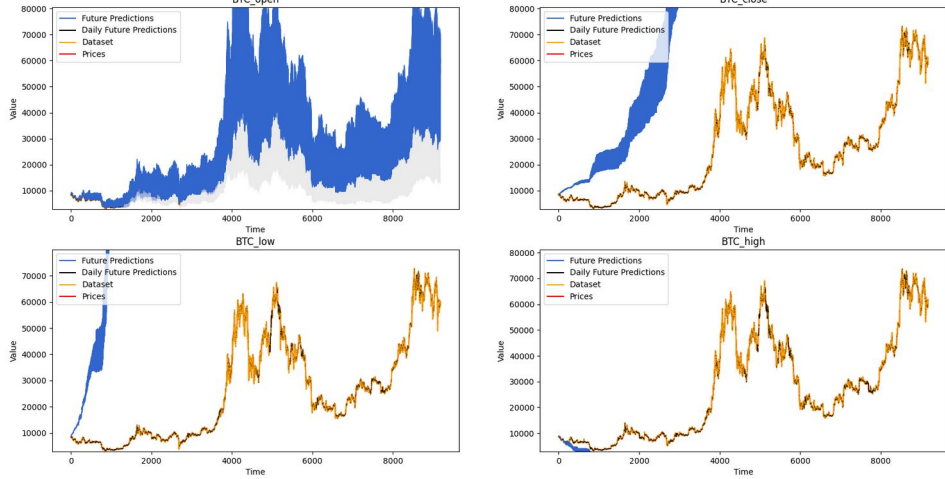


Distributions of predictions on the val dataset.
Blue: price_t+1 from price_t, Greens: price_t+1 from price_t where $i > 1$
All other plots check the temporal fit to the dataset, this plot checks overall ability to match real world $p(x)$



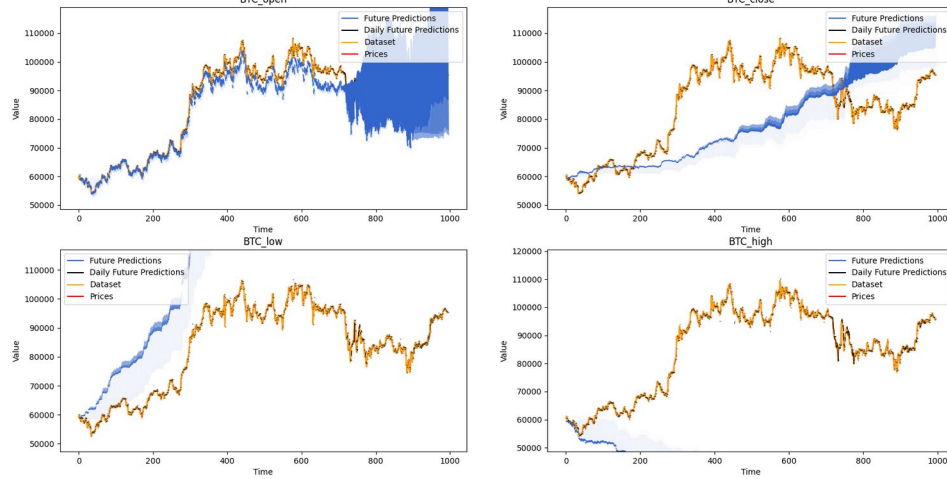
Predictions on the train dataset.

Red: real prices from API, Orange: re-scaled prices from dataset's normalization (should overlap with red)
 Blue: price $t+1$ from price t where price t is taken from previous prediction and >0 , fading away for further predictions
 Black: price $t+1$ from price t where price t is the real price of previous interval and >0 , fading away for further predictions



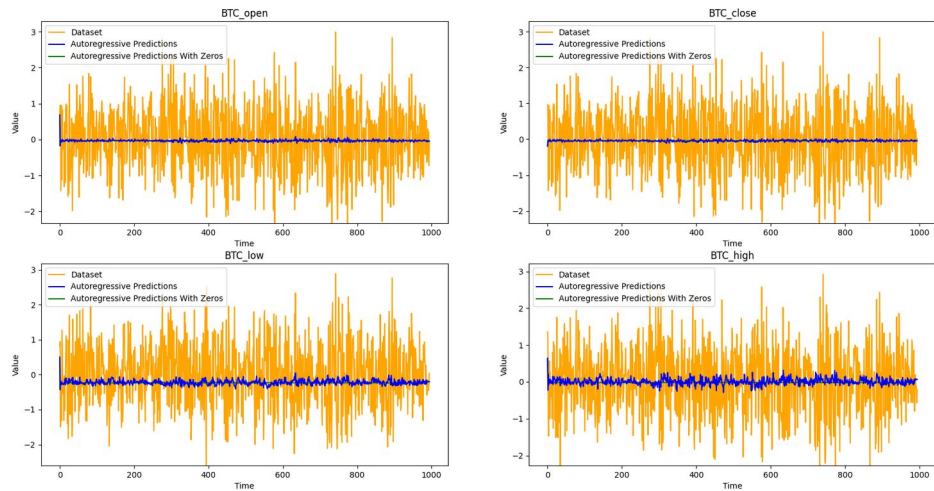
Predictions on the val dataset.

Red: real prices from API, Orange: re-scaled prices from dataset's normalization (should overlap with red)
 Blue: price $t+1$ from price t where price t is taken from previous prediction and >0 , fading away for further predictions
 Black: price $t+1$ from price t where price t is the real price of previous interval and >0 , fading away for further predictions



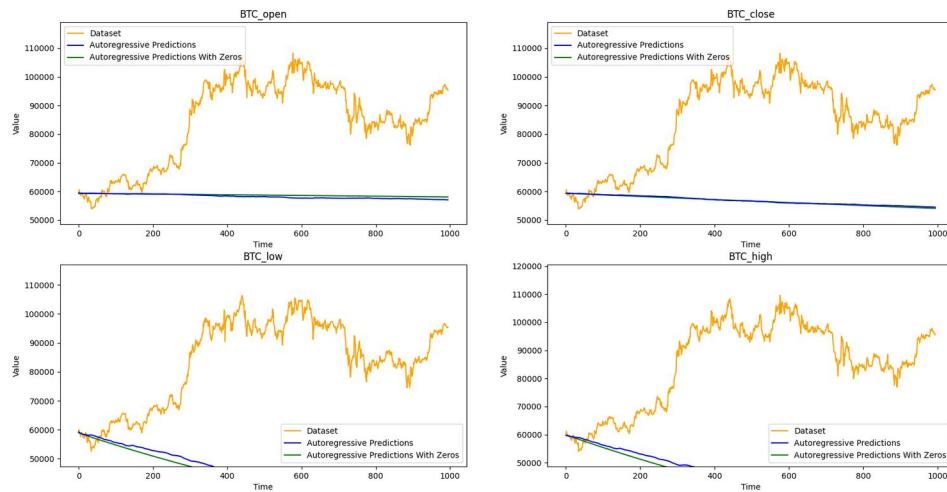
Autoregressive predictions on the val dataset.

Blue: filling the missing (non-predicted) input features from ground truth, Green: filling the missing (non-predicted) input features with zeros



Autoregressive predictions on the val dataset.

Blue: filling the missing (non-predicted) input features from ground truth, Green: filling the missing (non-predicted) input features with zeros

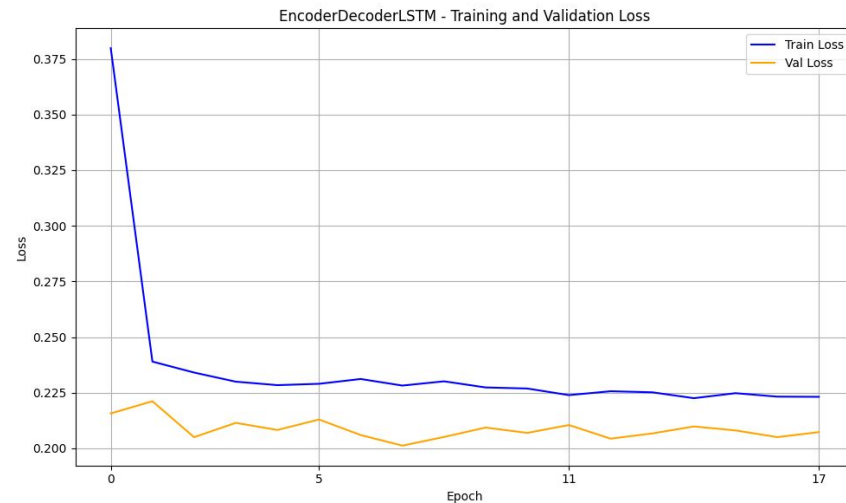
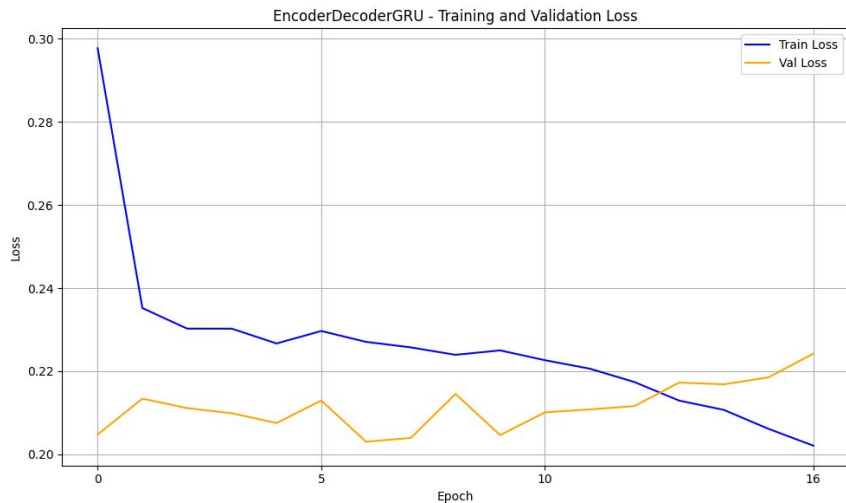


TRAINING: PHASE 1

In the end, I selected 2 (model, dataset) pair based on the loss values + plot results:

GRU & LSTM + IntervalLogReturnTransformLowHighRootCoinDataset

Looks like encoding the low and high signs in the normalization helped with the results.



TRADING STRATEGIES

Before continuing with the profit evaluation, we need some baselines. We have 14 benchmark agents:

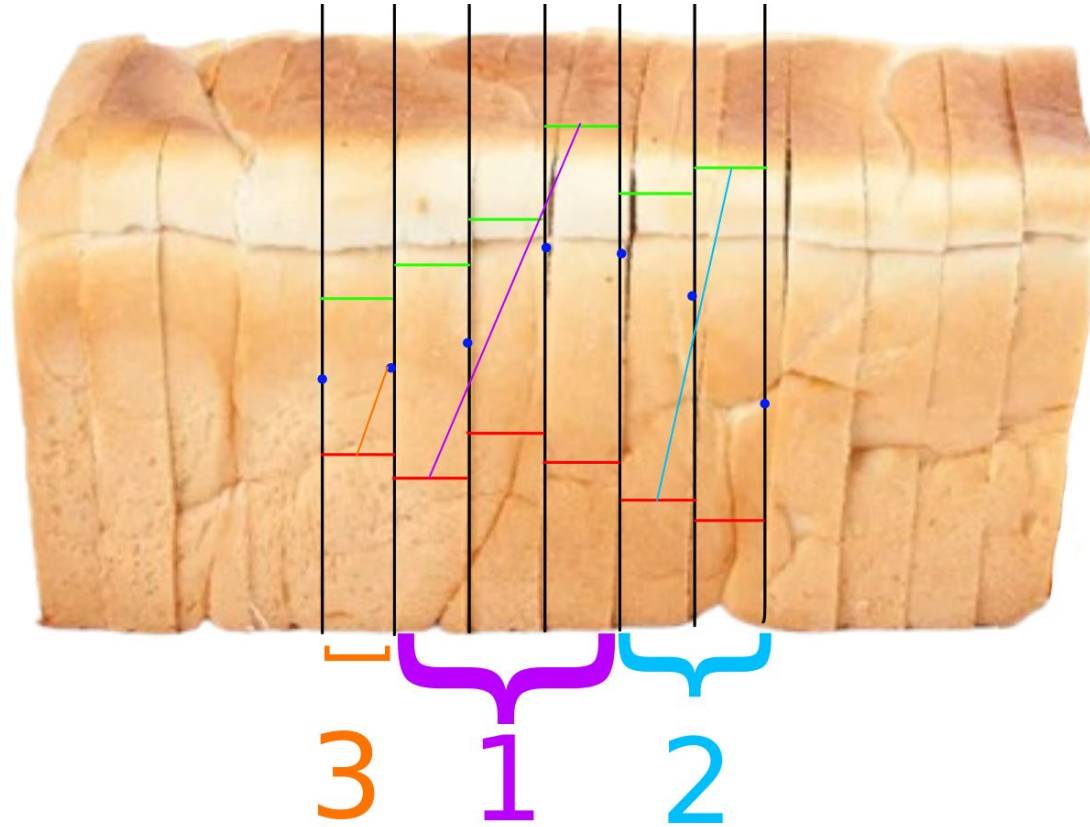
- random_all_in_agent
- random_portional_agent
- holder_buyer_seller_agent
- holder_agent
- martingale_agent
- dollar_cost_averaging_agent
- sma_crossover_agent
- volatility_agent*
- percentage_change_following_agent
- low_high_slope_agent*
- yesterday_trend_breaking_agent*
- mean_reversion_agent
- candlestick_pattern_agent*
- buy_yesterdays_low_sell_yesterdays_high_agent*

Strategies with * utilize the low-high features, where others don't plan buys and sells around low and high features.

TOAST BREAD

But all these strategies use the past data, we need a strategy that plans buys and sells around error-prone future price predictions to use with model. For this, I wrote a strategy called "Toast Bread", which creates and updates buy and sell orders optimally based on the future predictions.

TOAST BREAD



1. finds buy-sell orders spanning across multiple intervals (1 and 2), called "sandwiches"

2. calculates if there is a profitable buy-sell order in leftover single intervals (3), called "slices"

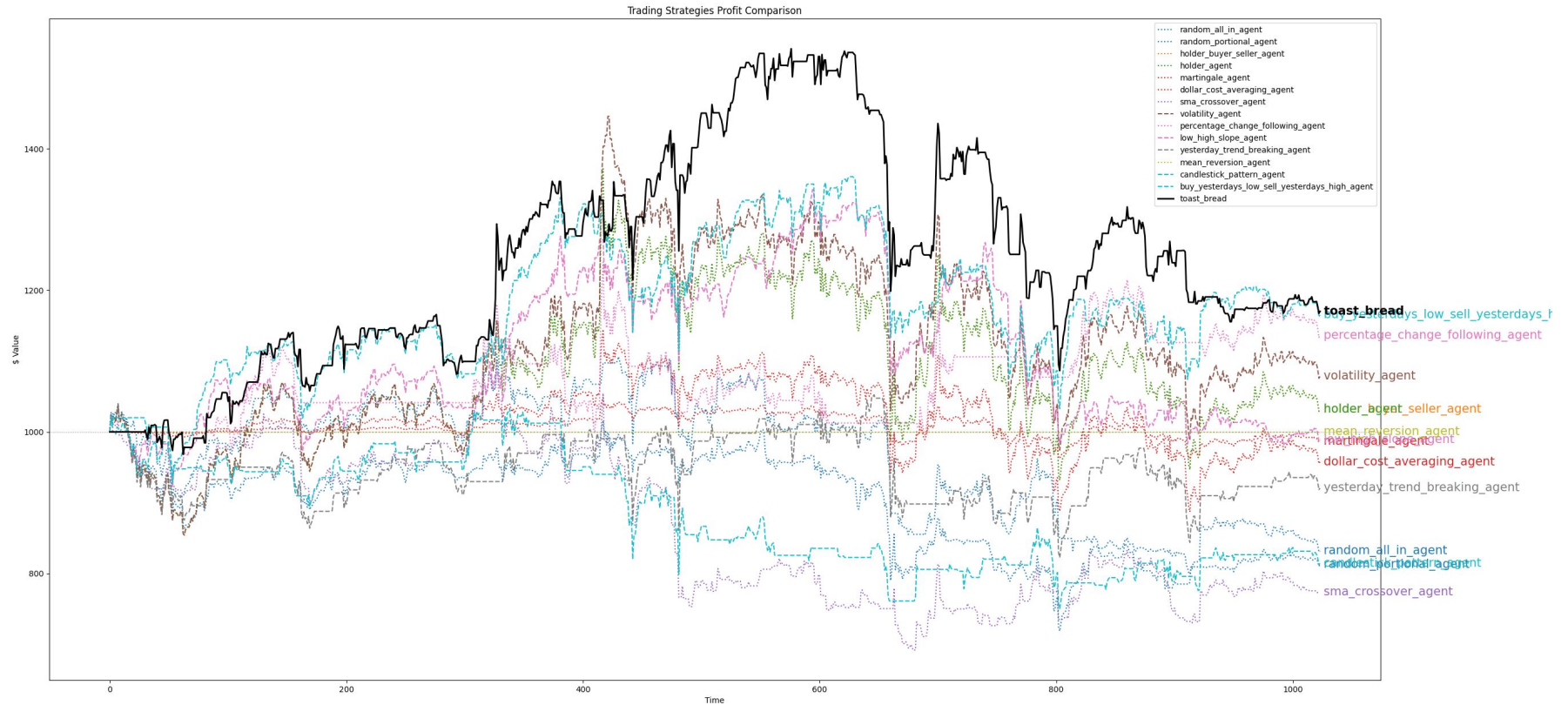
3. Starts executing with 3. When the next interval comes, discards 1 and 2, creates new buy-sell orders.

*Since we don't know the low-high order in intervals, this strategy ensures no buy-sell on low-high in the same interval

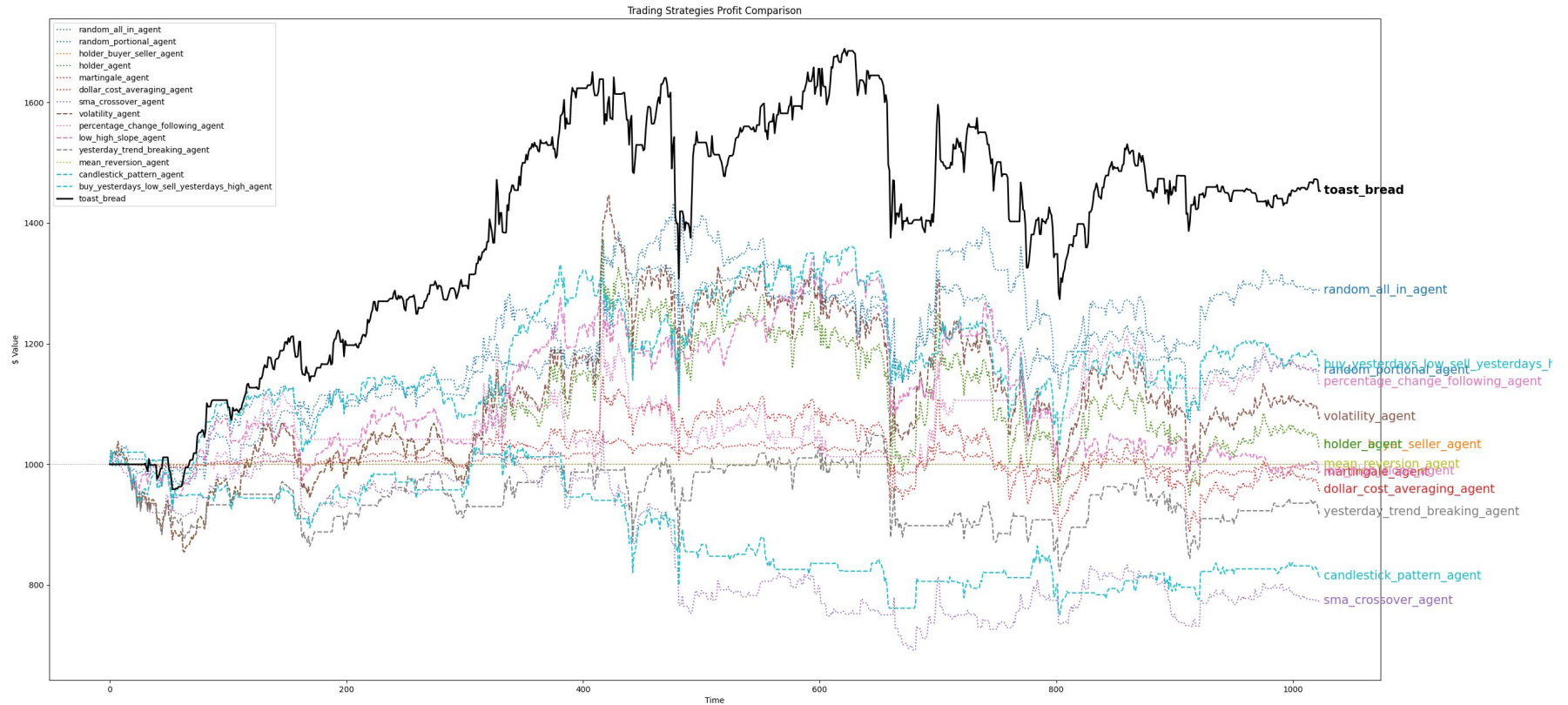
TRAINING: PHASE 2

After doing 8 new trainings (4 with GRU 4 with LSTM on all coins) with very base settings (no regularization, no augmentation etc.), and evaluating them with the trading strategies, here are the most promissing (of course cherrypicked) 3 models' performances:

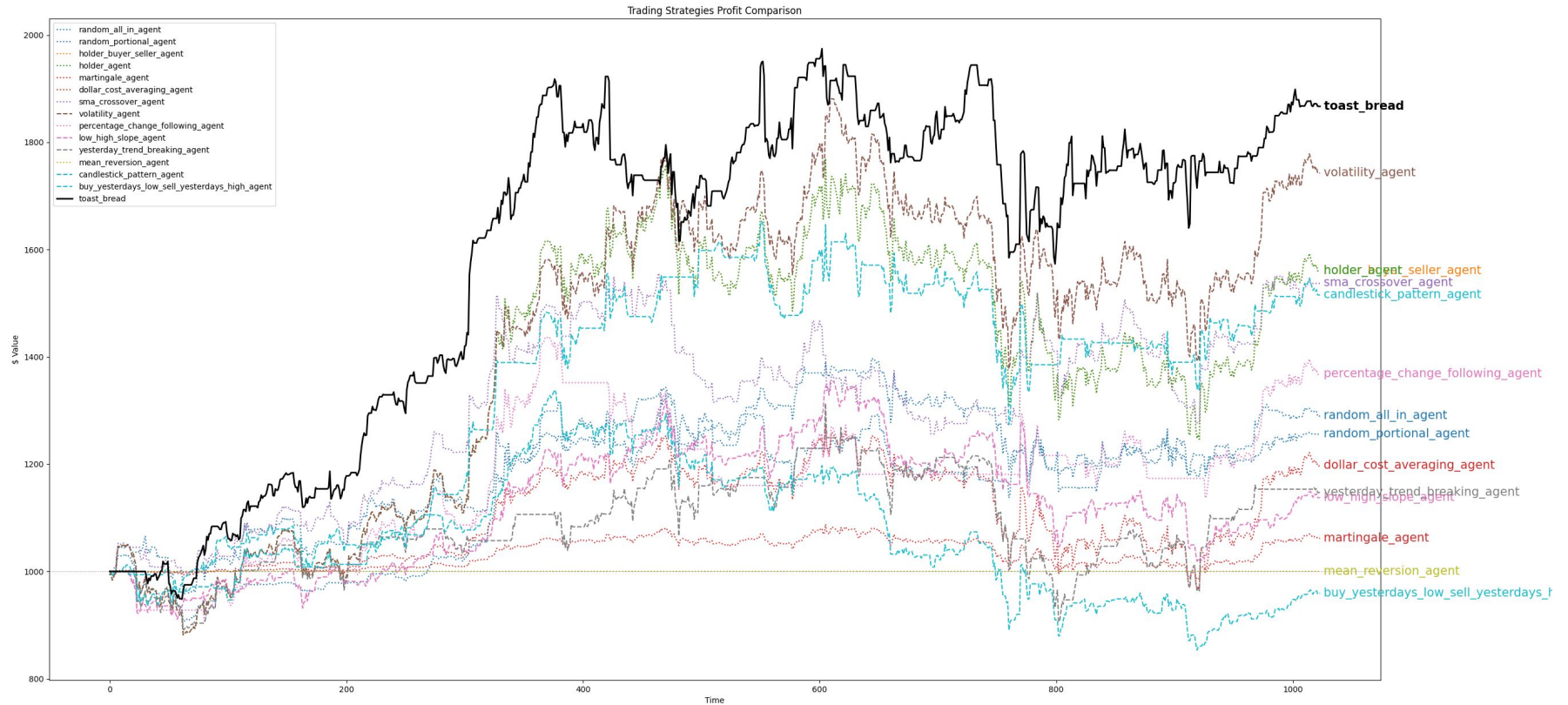
GRU ON BTC (~1.2x)



LSTM ON BNB (~1.4x)



LSTM ON BTC (~1.9x)



TRAINING: PHASE 3

Now since we found a trainable model (LSTM on BTC) with positive profit margin, we do random hyperparameter search to obtain 'final' models.

After 30+ trainings with LSTM on BTC, we collect the best 3 and continue to test with them. And compare with the benchmarks.

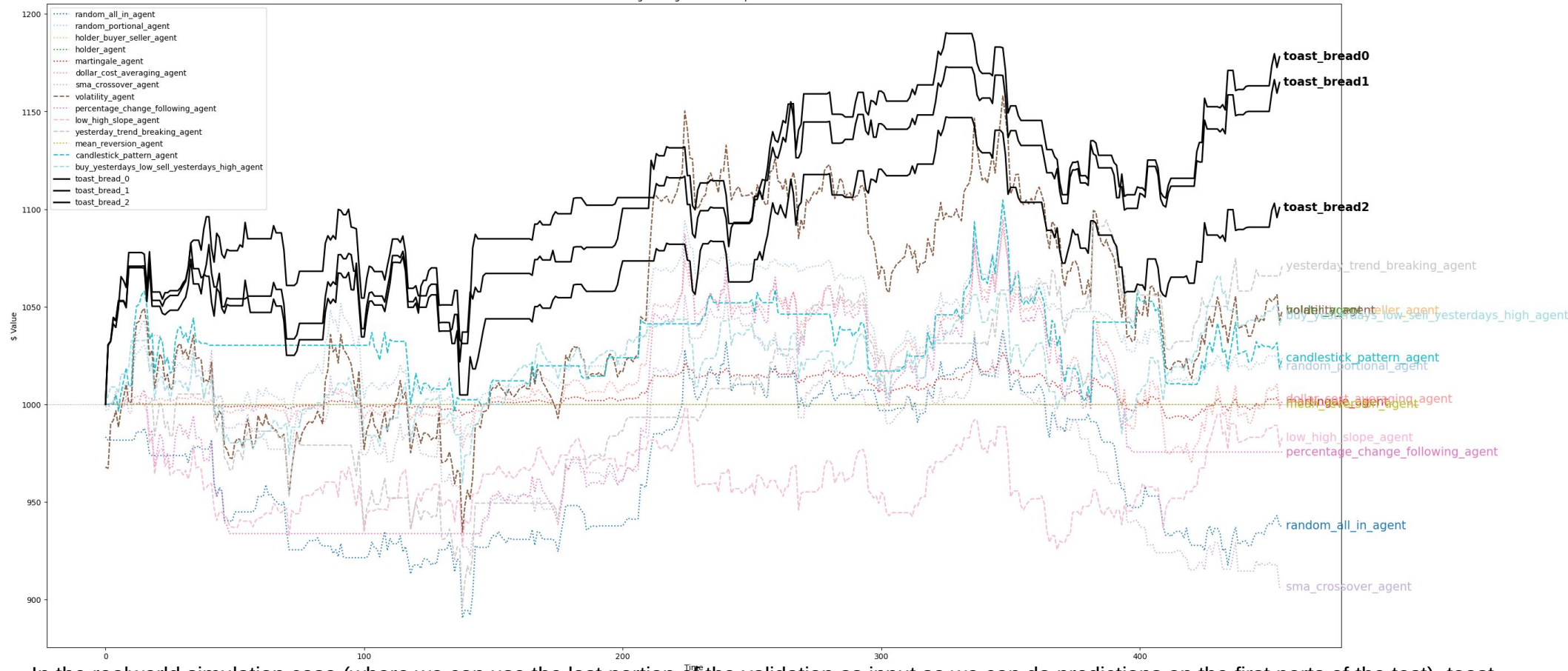
TESTING



Now we collect the test portion, (which ends up being almost 5% of the whole dataset) and test our selected models and compare performance.

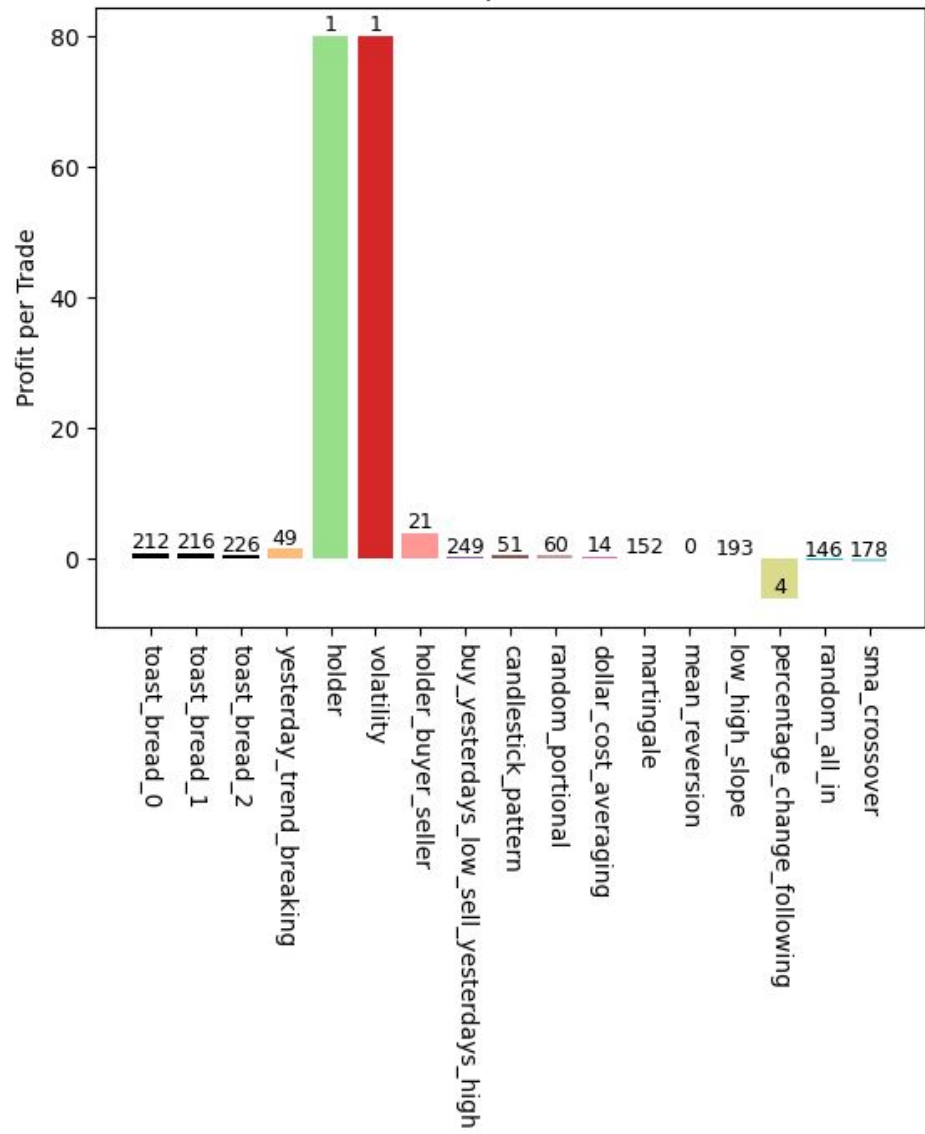
TESTING

Trading Strategies Profit Comparison

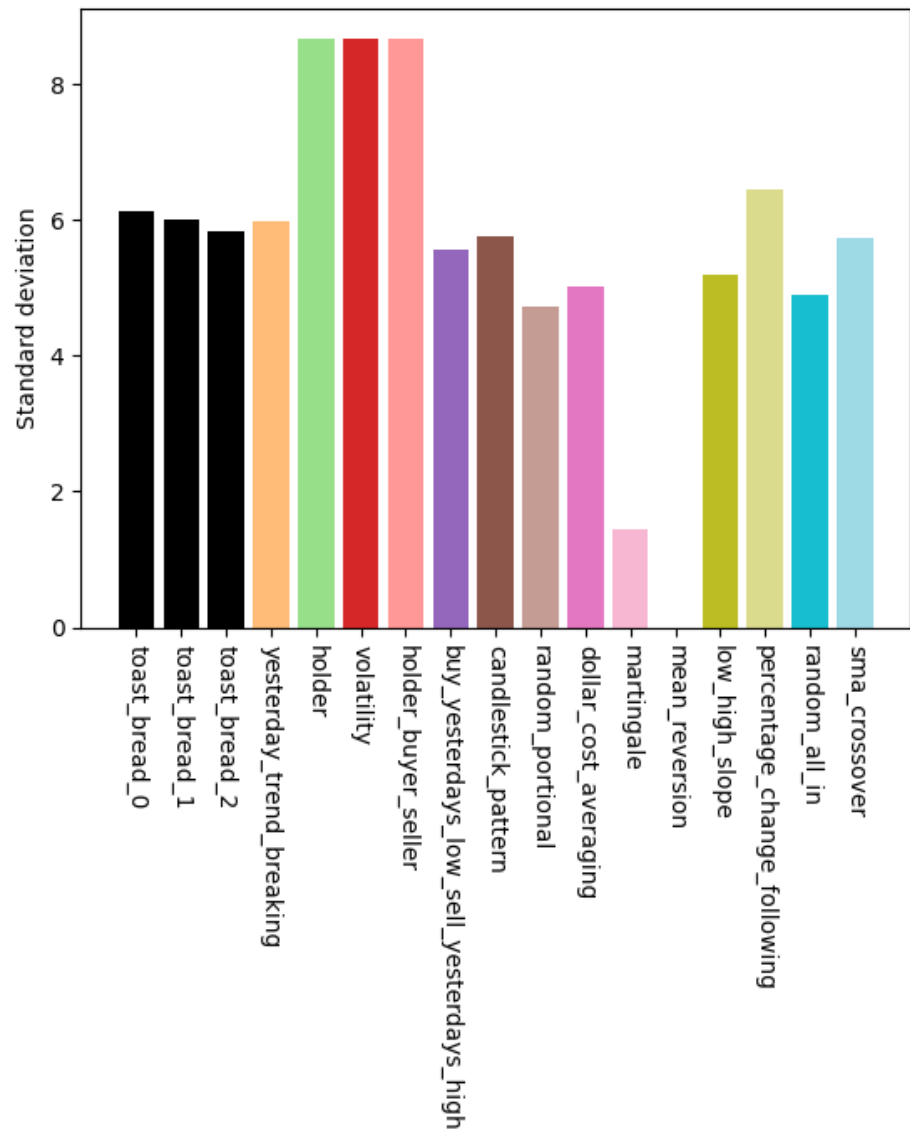


In the realworld simulation case (where we can use the last portion of the validation as input so we can do predictions on the first parts of the test) toast breads performed better. This case is excluded and test is kept entirely seperate from the rest of the dataset for scientific experimenting.

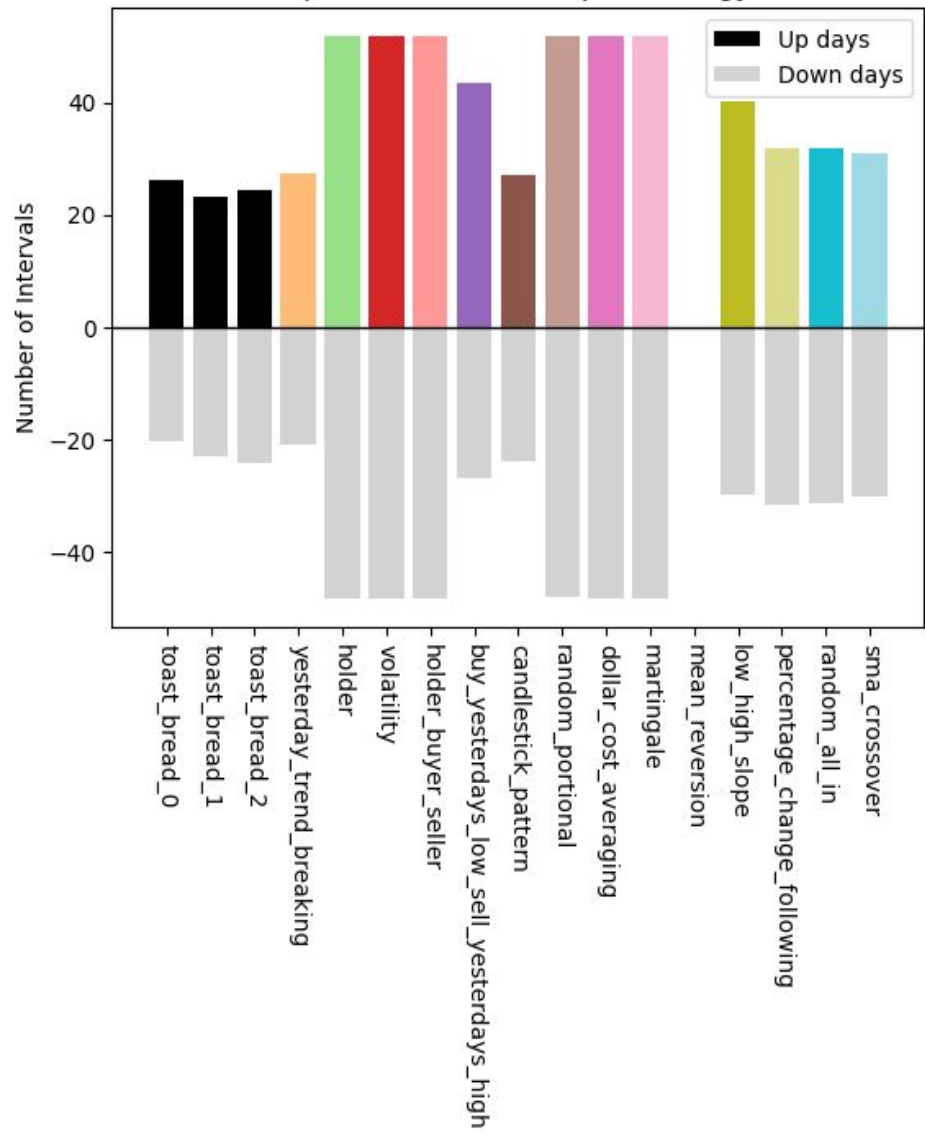
Profit per Trade



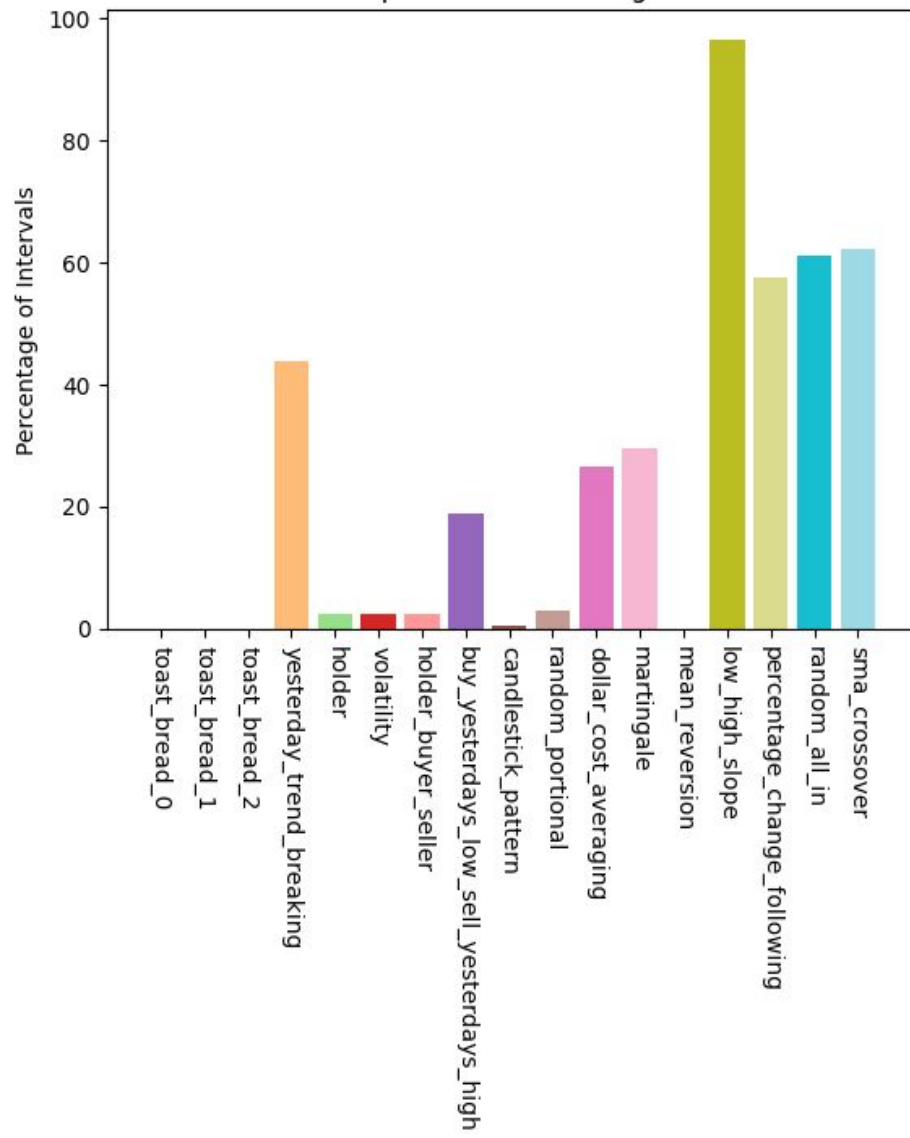
Standard deviation on interval values



Up vs Down Intervals per Strategy



Time Spent Below Starting Value



SHORTCOMINGS & TODOs FOR FUTURE

- 1-More coins (litecoin etc.) and features (like volume)
- 2-Invertable finance preprocessings like Fourier, Wavelet etc.
- 3-Dataset, model pairs that can use high-low order
- 4-New trading strategies that utilize both future predictions and past data to decide on a buy/sell order

Additional Notes on Low-High Order

I actually tried multiple different approaches to include order information in the pipeline. Knowing whether price first peaked and then dipped or vice-versa could be the most deciding information for the future predictions.

- I tried to hint it with hand crafted features like trajectory and stability
- I tried to set up datasets that tell the order of the low-high without any additional crafted features
- I manipulated intervals, merging them and creating new low-high ordered intervals

But they either failed, not improved the performance or was logically not perfect to include the pipeline because of the stacked normalizations applied to the data. I decided to not include them in the presentation for the sake of staying in the allocated time.